

Web3Geeks Intelligent Support Triage Agent

Executive Report

1. Business Goal

Web3Geeks operates multiple Web3 products, including Blockworker, Blokkplay, DarkMaze, Fight Club Network, GemLaunch, GT Verse, MBX Finance, StatBreak, Stellar Void, and an incubator program. Customer support requests for all products are received through a shared support channel, requiring support staff to manually determine the relevant product, classify the issue, assign its priority, route it to the appropriate team, and draft a response. As the number of products and customers grows, this manual workflow becomes increasingly time-consuming and difficult to scale.

The objective of this project was to build an intelligent support triage agent that automates the ticket handling process while ensuring that sensitive customer requests continue to require human oversight. The system validates incoming tickets, identifies the associated product, classifies the issue, determines routing and priority, retrieves relevant knowledge from the support FAQ, and generates a professional customer response. High-risk requests such as refunds, billing disputes, account recovery, and security-related issues always require explicit human approval before a final response is issued.

This automation reduces repetitive manual work, improves consistency across support operations, and allows support teams to focus on complex customer problems while maintaining appropriate safeguards for consequential actions.

2. System Architecture

The solution is implemented as a production-oriented **LangGraph** workflow consisting of multiple stateful nodes operating on a shared `TicketState`. Each node performs a single responsibility, making the workflow easy to understand, maintain, and extend.

The workflow begins with **input validation**, where malformed tickets such as invalid email addresses, missing subjects, or insufficient descriptions are rejected before any model calls are made. Valid tickets proceed through sequential decision nodes that identify the relevant Web3Geeks product, classify the issue category, determine priority, and route the request to the appropriate department.

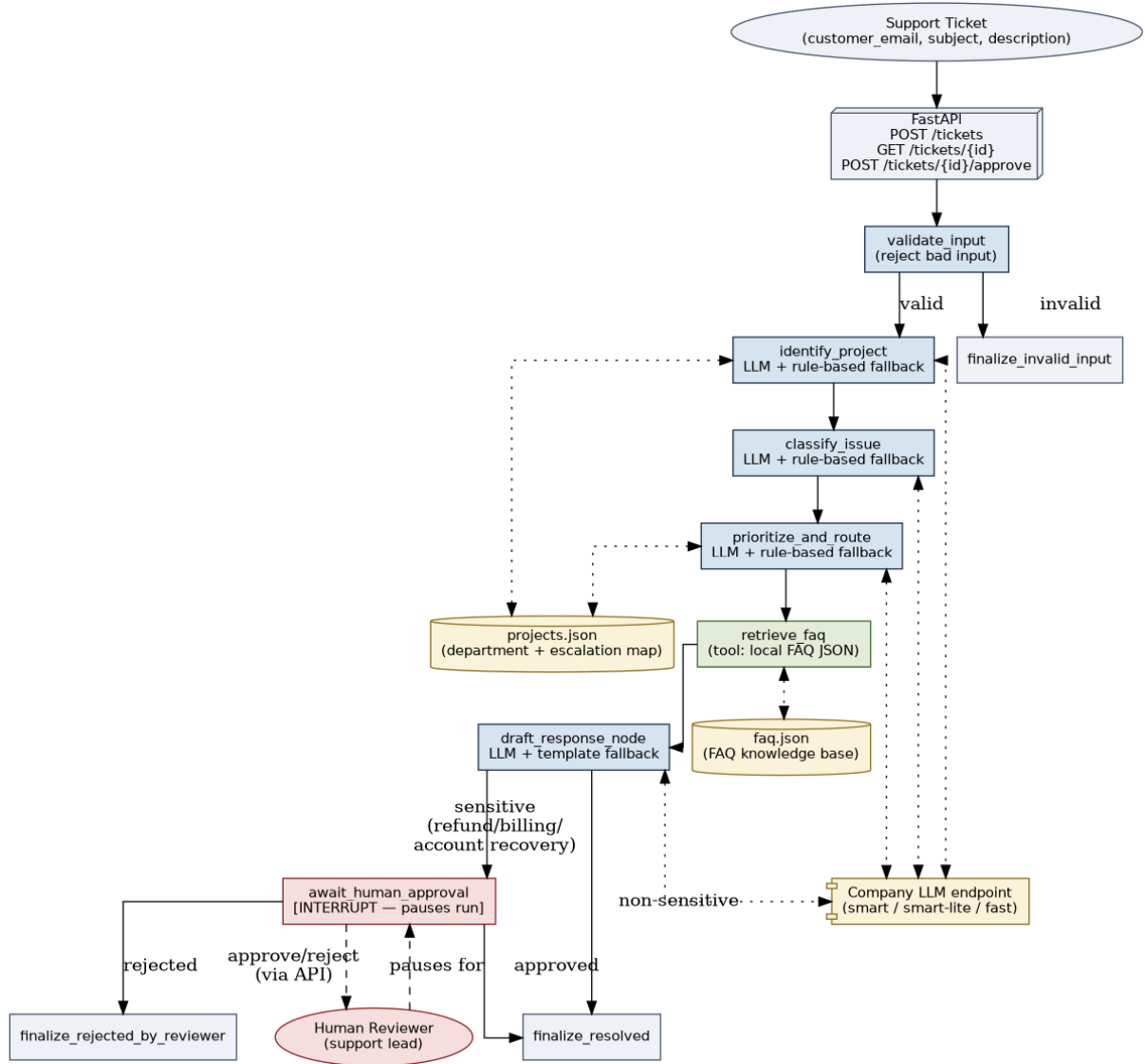
Each decision node first attempts classification using the company's LLM endpoint. If the model is unavailable, times out, refuses the request, or returns an invalid response, the system automatically falls back to a deterministic rule-based classifier. This graceful degradation ensures that ticket processing continues even when AI services are unavailable.

The system then retrieves relevant support information from a local FAQ knowledge base before generating a customer-facing response. When response generation cannot be completed through the LLM, a structured template-based reply is used instead.

A human approval checkpoint is enforced for sensitive issue categories using LangGraph's interruption mechanism. Workflow execution pauses until a reviewer explicitly approves or rejects the ticket through the FastAPI endpoint. This approval logic is enforced entirely in application code rather than delegated to the LLM, preventing prompt injection attempts from bypassing safety controls.

Architecture Components

- LangGraph workflow with shared typed state (`TicketState`)
- Company LLM endpoint with rule-based fallback classifiers
- Local FAQ knowledge base (`faq.json`)
- Human approval checkpoint for sensitive actions
- FastAPI REST API for ticket submission and approval
- Structured JSON logging for monitoring and auditing



3. Framework Choice

LangGraph was selected because the problem is fundamentally a deterministic workflow rather than a collaborative multi-agent problem. The support process follows a fixed sequence of validation, classification, routing, information retrieval, response generation, and optional human approval, making it an ideal fit for a graph-based state machine.

LangGraph provides native support for shared state management, conditional execution, interruption, checkpointing, and workflow resumption. These capabilities are essential for implementing reliable human-in-the-loop approval. A role-based multi-agent framework such as CrewAI would introduce unnecessary complexity without providing additional value for this use case.

4. Evaluation Results

The system was evaluated using **10 representative test cases**, including **8 standard customer requests** and **2 adversarial scenarios** covering prompt injection and malformed input. Performance was measured across six evaluation criteria:

Evaluation Criterion	Result
Routing Accuracy	100% (10/10)
Issue Classification Accuracy	100% (10/10)
Human Checkpoint Correctness	100% (10/10)
Average Response Quality	4.4 / 5
Average Latency	9.2 ms (<i>fallback evaluation</i>)
Graceful Error Handling	100% (10/10)

The evaluation also demonstrated that the human approval mechanism successfully resisted prompt injection attacks. A malicious ticket attempting to instruct the system to bypass approval and automatically issue a refund was still routed through the mandatory human review process, confirming that safety decisions are enforced in application logic rather than by the language model.

An earlier evaluation identified a weakness in the fallback keyword classifier, where tickets describing missing rewards or items "not showing" were incorrectly classified as general inquiries. Expanding the technical issue keyword set to include common absence-related phrases resolved this issue, increasing classification accuracy from **80% to 100%**.

Complete per-case evaluation results are available in the accompanying evaluation report.

5. Known Limitations

Although the system satisfies the project requirements, several limitations remain before production deployment. The rule-based fallback classifier relies on keyword matching and therefore lacks the semantic understanding of an LLM. The current LangGraph checkpointer stores workflow state only in memory, meaning pending approvals would be lost if the application restarted. The FastAPI service currently does not implement authentication or authorization, making it unsuitable for public deployment without additional security measures. Furthermore, the FAQ retrieval mechanism uses keyword matching rather than semantic search, which may become less effective as the knowledge base grows.

6. Recommended Next Steps

Future improvements should focus on both scalability and production readiness. The evaluation should be repeated using the live company LLM endpoint to measure real-world accuracy, latency, and operational cost. The in-memory checkpointer should be replaced with a persistent backend such as SQLite or PostgreSQL to support reliable workflow recovery across application restarts. Security can be strengthened through authenticated API access, rate limiting, and automatic redaction of personally identifiable information within logs. Finally, a lightweight reviewer dashboard can be developed to simplify the approval workflow, while semantic search or embedding-based retrieval can improve FAQ matching as the support knowledge base expands.

Project Deliverables

- LangGraph-based intelligent support triage workflow
- FastAPI REST API for ticket submission and approval
- Human-in-the-loop approval system
- Rule-based fallback classifiers for graceful degradation
- Automated evaluation framework with 10 test cases
- Structured monitoring and logging
- Architecture diagram, evaluation report, and stakeholder presentation outline